

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №8**

**«Мережеве та асинхронне програмування. REST API на FastAPI, asyncio та
HTTPX»**

з курсу «Професійний Python»

Варіант №1 — «Асинхронний REST API системи обліку студентів»

Виконав: Лис Артур

Група: ФЕП-31

Перевірів(ла): _____

Дата: 08.09.2026

Львів — 2026

1. Мета роботи

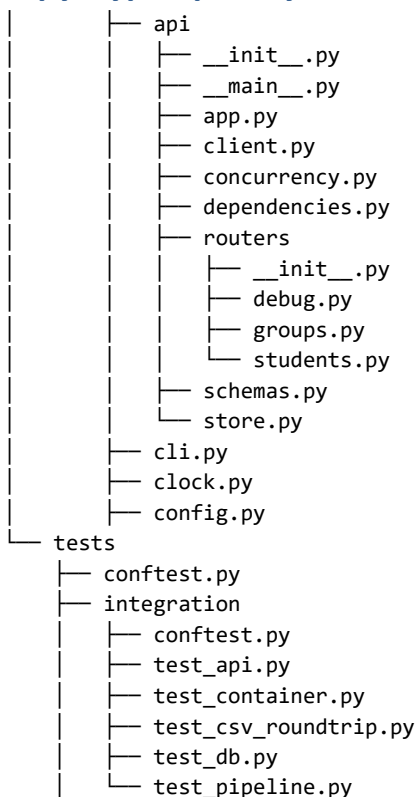
Створити асинхронний REST API для проєкту обліку студентів: описати ресурси `students` і `groups`, валідувати дані Pydantic-схемами, повертати коректні коди стану, підключити persistence layer JP7, застосувати `async/await`, `asyncio.Task`, `gather()`, `timeout`, `retry` з `backoff` і `Semaphore`, звертатися до API через `httpx.AsyncClient` і покрити все автоматизованими API-тестами.

2. Індивідуальний варіант і постановка задачі

Варіант 1 — система обліку студентів. Обов'язкові endpoint-и: `GET /students`, `GET /students/{id}`, `POST /students`, `PATCH /students/{id}`, `DELETE /students/{id}`, додатково `GET /groups/{group}/students` та `GET /groups/{group}/statistics`. API має додавати, переглядати, шукати за прізвищем, фільтрувати за групою, змінювати бал, переводити до іншої групи, видаляти, рахувати середній бал групи та визначати найкращого студента. Асинхронне завдання — одночасно отримати статистику кількох груп через `gather()` з обмеженням через `Semaphore`.

Понад мінімум: пагінація, сортування, кілька query-параметрів, структуровані помилки через `custom exception handlers`, `dependency injection`, батч-endpoint `/groups/statistics`, паралельні HTTP-запити, `exponential backoff`, налаштовувані `timeout` і кількість спроб, OpenAPI-метадані, `lifecycle`-керування `AsyncClient`, `mock` зовнішнього HTTP-сервісу в тестах (`httpx.MockTransport`). Групу в шляху ідентифіковано кодом (ФЕП-31с), а не числовим `id` — це природний ключ таблиці `groups` з JP7.

3. Структура проєкту



```

└─ unit
    ├── test_analytics.py
    ├── test_config.py
    ├── test_domain.py
    ├── test_models.py
    ├── test_notify.py
    ├── test_registry.py
    ├── test_snapshot.py
    ├── test_storage.py
    └── test_streaming.py

```

`schemas.py` — Pydantic-моделі запитів і відповідей; `store.py` — репозиторії LP7, загорнуті в операції, що повертають схеми; `routers/` — endpoint-и; `app.py` — фабрика застосунку з `lifespan` та обробниками винятків; `concurrency.py` — `Task/gather/Semaphore/timeout/retry`; `client.py` — HTTPX-клієнт; `__main__.py` — команди `serve` (Uvicorn) і `demo`.

4. Опис REST API

Метод	Шлях	Параметри	Код	Призначення
GET	/students	query: last_name, group, sort, page, size	200	список із пошуком, фільтром, пагінацією
GET	/students/best	query: group	200/404	найкращий студент (загалом або в групі)
GET	/students/{id}	path: id	200/404	один студент
POST	/students	body: StudentCreate	201/409/422	додати студента
PATCH	/students/{id}	path: id; body: StudentPatch	200/404/409/422	бал, переведення, email
DELETE	/students/{id}	path: id	204/404	видалити
GET	/groups	—	200	усі групи зі статистикою
GET	/groups/statistics	query: code (повторюваний), limit	200/404	кілька груп одночасно (gather)
GET	/groups/{code}/students	path: code	200/404	студенти групи
GET	/groups/{code}/statistics	path: code	200/404	кількість, середній бал, найкращий

Path-параметри — частина шляху (/students/7, /groups/ФЕП-31c/students); query-параметри уточнюють вибірку (/students?group=ФЕП-32&sort=-grade&size=1). Коди: 200 — успіх, 201 — створено, 204 — видалено без тіла, 404 — ресурсу немає, 409 — конфлікт з обмеженням бази, 422 — тіло або параметри не пройшли валідацію.

5. Pydantic-схеми та правила валідації

```
Grade = Annotated[float, Field(ge=GRADE_MIN, le=GRADE_MAX, examples=[88.5], description="Середній бал, 0..100")]
GroupCode = Annotated[str, Field(pattern=f"^{GROUP_PATTERN}$", examples=["ФЕП-31c"], description="Код групи")]
Name = Annotated[str, Field(min_length=1, max_length=64, examples=["Марта"])]
Email = Annotated[str, Field(pattern=r"^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$", max_length=120, examples=["marta@lnu.edu.ua"])]
```

```
class StudentCreate(BaseModel):
    """POST /students body. Unknown fields are rejected (extra="forbid"), strings are stripped."""
```

```
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
```

```
    first_name: Name
    last_name: Name
    group: GroupCode
    average_grade: Grade
    email: Email | None = None
```

```
class StudentPatch(BaseModel):
    """PATCH /students/{id} body: every field optional, at least one must be present."""
```

```
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
```

```
    average_grade: Grade | None = None
    group: GroupCode | None = None
    email: Email | None = None
```

```
    def is_empty(self) -> bool:
        return not self.model_fields_set
```

Правила збігаються з моделлю ЛР1: бал у межах 0..100, код групи за тим самим регулярним виразом, непорожні імена, email за шаблоном. extra="forbid" відхиляє невідомі поля, str_strip_whitespace прибирає пробіли. StudentPatch робить кожне поле необов'язковим, а is_empty() дозволяє повернути 422 на порожній PATCH. StudentOut — response model: FastAPI серіалізує лише його поля, тож внутрішні деталі ORM назовні не потрапляють.

6. FastAPI-застосунок

```
def create_app(*, database_url: str = DEFAULT_URL, include_debug: bool = False) -> FastAPI:
    @asynccontextmanager
```

```

async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    engine = make_engine(database_url)
    create_schema(engine) # no-op on a database already built by `alembic upgrade head`
    app.state.store = StudentStore(make_session_factory(engine))
    app.state.http = httpx.AsyncClient(timeout=5.0) # one outbound client for the whole app
    lifetime
    app.state.flaky_calls = 0
    try:
        yield
    finally:
        await app.state.http.aclose()
        engine.dispose()

app = FastAPI(title="Student Manager API", version="0.8.0", description=DESCRIPTION,
openapi_tags=TAGS,
            lifespan=lifespan, contact={"name": "Артур Лис", "email": "Artur.Lys@lnu.edu.ua"})
app.include_router(students.router)
app.include_router(groups.router)
if include_debug:
    app.include_router(debug.router)
register_exception_handlers(app)
return app

@app.exception_handler(RecordNotFoundError)
async def not_found(_: Request, exc: RecordNotFoundError) -> JSONResponse:
    return error(404, "not_found", exc)

@app.exception_handler(ConstraintViolationError)
async def conflict(_: Request, exc: ConstraintViolationError) -> JSONResponse:
    return error(409, "conflict", exc)

```

Lifespan створює engine, StudentStore і один вихідний httpx.AsyncClient на весь час життя застосунку та закриває їх при зупинці. Endpoint-и отримують store через Depends(get_store). Доменні винятки LP7 перетворюються на JSON {error, detail} із правильним кодом — traceback клієнтові не потрапляє.

7. Endpoint-и: GET, POST, PATCH, DELETE

```

@router.get("", response_model=StudentPage, summary="Список студентів із пошуком, фільтром і пагінацією")
async def list_students(
    store: Store,
    last_name: Annotated[str | None, Query(min_length=1, description="Префікс прізвища, без урахування регістру")] = None,
    group: Annotated[GroupCode | None, Query(description="Лише студенти цієї групи")] = None,
    page: Annotated[int, Query(ge=1)] = 1,
    size: Annotated[int, Query(ge=1, le=100)] = 20,
    sort: SortKey = "id",
) -> StudentPage:
    return await asyncio.to_thread(store.search, last_name=last_name, group=group, page=page, size=size, sort=sort)

@router.post("", response_model=StudentOut, status_code=status.HTTP_201_CREATED,
            responses={409: {"model": ErrorOut, "description": "Порушення обмеження (наприклад, email зайнято)}}})
async def create_student(store: Store, data: StudentCreate) -> StudentOut:

```

```

        return await asyncio.to_thread(store.create, data)

@router.patch("/{student_id}", response_model=StudentOut, responses=NOT_FOUND,
              summary="Змінити бал, перевести до іншої групи або оновити email")
async def patch_student(store: Store, student_id: int, data: StudentPatch) -> StudentOut:
    if data.is_empty():
        raise HTTPException(status.HTTP_422_UNPROCESSABLE_CONTENT, detail="тіло PATCH порожнє: немає що змінювати")
    return await asyncio.to_thread(store.patch, student_id, data)

@router.delete("/{student_id}", status_code=status.HTTP_204_NO_CONTENT, responses=NOT_FOUND)
async def delete_student(store: Store, student_id: int) -> Response:
    if not await asyncio.to_thread(store.delete, student_id):
        raise HTTPException(status.HTTP_404_NOT_FOUND, detail=f"студента з id={student_id} не знайдено")
    return Response(status_code=status.HTTP_204_NO_CONTENT)

```

Кожен обробник — `async def` із `await`: сесія SQLAlchemy синхронна, тому виклик `store` виконується у робочому потоці через `asyncio.to_thread`, а `event loop` тим часом обслуговує інші запити. `store.search` буде один `SELECT` з `JOIN`, умовами за префік прізвища та групою, `COUNT` для `total`, `ORDER BY` і `OFFSET/LIMIT`.

```

def search(self, *, last_name: str | None = None, group: str | None = None, page: int = 1, size:
int = 20,
          sort: SortKey = "id") -> StudentPage:
    with self._transaction() as session:
        statement =
select(StudentRecord).join(StudentRecord.group).options(contains_eager(StudentRecord.group))
        if last_name:
            statement =
statement.where(func.casefold(StudentRecord.last_name).like(f"{last_name.casefold()}%"))
        if group:
            statement = statement.where(GroupRecord.code == group)
        total = session.scalar(select(func.count()).select_from(statement.subquery())) or 0
        order = {
            "id": StudentRecord.id, "last_name": StudentRecord.last_name,
            "grade": StudentRecord.average_grade, "-grade": StudentRecord.average_grade.desc(),
        }[sort]
        rows = session.scalars(statement.order_by(order, StudentRecord.id).offset((page - 1) *
size).limit(size))
        return StudentPage(items=[StudentOut.from_record(r) for r in rows], total=total, page=page,
size=size)

```

8. Приклади JSON-запитів і відповідей

POST /students (201) і validation (422)

```

> POST /students json={"first_name": "Марта", "last_name": "Гнатишин", "group": "ФЕП-31с",
"average_grade": 93.4, "email": "marta@lnu.edu.ua"}
< 201 Created
{
  "id": 1,
  "first_name": "Марта",
  "last_name": "Гнатишин",
  "full_name": "Гнатишин Марта",

```

```

    "group": "ФЕП-31с",
    "average_grade": 93.4,
    "email": "marta@lnu.edu.ua"
}

> POST /students json={"first_name": "Іроп", "last_name": "Новак", "group": "фен31", "average_grade":
150}
< 422 Unprocessable Entity
{
  "detail": [
    {
      "type": "string_pattern_mismatch",
      "loc": [
        "body",
        "group"
      ],
      "msg": "String should match pattern '^[А-ЯІЇЄґА-Z]{2,5}-\\d{2}[а-яіїєґа-з]?$',",
      "input": "фен31",
      "ctx": {
        "pattern": "^[А-ЯІЇЄґА-Z]{2,5}-\\d{2}[а-яіїєґа-з]?$"
      }
    },
    {
      "type": "less_than_equal",
      "loc": [
        "body",
        "average_grade"
      ],
      "msg": "Input should be less than or equal to 100",
      "input": 150,
      "ctx": {
        "le": 100.0
      }
    }
  ]
}

```

GET /students/{id}, /students/best, 404

```

-----
> GET /students/1
< 200 OK
{
  "id": 1,
  "first_name": "Марта",
  "last_name": "Гнатишин",
  "full_name": "Гнатишин Марта",
  "group": "ФЕП-31с",
  "average_grade": 93.4,
  "email": "marta@lnu.edu.ua"
}
> GET /students/best
< 200 OK
{
  "id": 3,
  "first_name": "Соломія",
  "last_name": "Кравець",
  "full_name": "Кравець Соломія",
  "group": "ФЕП-32",
  "average_grade": 88.1,

```

```

    "email": null
}
> GET /students/99
< 404 Not Found
{
    "error": "not_found",
    "detail": "у таблиці students немає запису з id=99"
}

> PATCH /students/3 json={"email": "marta@lnu.edu.ua"}
< 409 Conflict
{
    "error": "conflict",
    "detail": "база даних відхилила зміну: UNIQUE constraint failed: students.email"
}

DELETE /students/{id}: 204, потім 404
-----
> DELETE /students/3
< 204 No Content
> DELETE /students/3
< 404 Not Found
{
    "detail": "студента з id=3 не знайдено"
}

```

9. asyncio: Task, gather, timeout, Semaphore, retry

```

async def gather_statistics(source: StatisticsSource, codes: Iterable[str], *, limit: int = 3,
                           timeout: float = 2.0) -> list[GroupStatisticsOut]:
    """Statistics of several groups at once.

```

One Task per group; the Semaphore lets at most ``limit`` of them touch the database simultaneously; each

database call runs in a worker thread (the SQLAlchemy session is synchronous) under its own timeout.

``gather`` keeps the order of ``codes`` and propagates the first exception.

```

    """
    semaphore = asyncio.Semaphore(limit)

```

```

    async def one(code: str) -> GroupStatisticsOut:
        async with semaphore:
            async with asyncio.timeout(timeout):
                return await asyncio.to_thread(source.group_statistics, code)

```

```

    tasks = [asyncio.create_task(one(code), name=f"statistics:{code}") for code in codes]
    return list(await asyncio.gather(*tasks))

```

```

async def retry(operation: Callable[[], Awaitable[T]], *, attempts: int = 3, base_delay: float = 0.2,
                retry_on: tuple[type[BaseException], ...] = (Exception,), label: str = "operation") ->

```

T:

"""Call ``operation`` up to ``attempts`` times; wait $\text{base_delay} * 2^{(n-1)}$ between attempts (exponential backoff)."""

```

    for attempt in range(1, attempts + 1):
        try:
            return await operation()
        except retry_on as error:

```

```

        reason = f"{type(error).__name__}: {error}" if str(error) else type(error).__name__
        if attempt == attempts:
            logger.warning("%s: спроба %d/%d невдала (%s) – здаюся", label, attempt, attempts,
reason)
            raise
            delay = base_delay * 2 ** (attempt - 1)
            logger.warning("%s: спроба %d/%d невдала (%s), повтор через %.2f с", label, attempt,
attempts, delay)
            await asyncio.sleep(delay)
            raise AssertionError("unreachable")

```

`gather_statistics` створює по одному Task на групу; семафор пускає до бази не більше `limit` задач одночасно, `asyncio.timeout` обмежує кожну; `gather` повертає результати в порядку кодів і прокидає перший виняток. `retry` повторює операцію з паузами 0.2, 0.4, 0.8 с (exponential backoff) і здається після останньої спроби. Endpoint `/groups/statistics?code=...&code=...` — це batch-варіант, який викликає `gather_statistics` напряму:

```

@router.get("/statistics", response_model=list[GroupStatisticsOut], responses=NOT_FOUND,
            summary="Статистика кількох груп одночасно (Task + gather + Semaphore)")
async def statistics_many(
    store: Store,
    code: Annotated[list[GroupCode], Query(min_length=1, description="Повторюваний параметр:
?code=A&code=B")],
    limit: Annotated[int, Query(ge=1, le=10, description="Скільки груп рахувати паралельно")] = 3,
) -> list[GroupStatisticsOut]:
    return await gather_statistics(store, code, limit=limit)

```

10. HTTPX-клієнт

```

class StudentApiClient:
    def __init__(self, base_url: str = "http://127.0.0.1:8000", *, timeout: float = 5.0, attempts: int
= 3,
                    base_delay: float = 0.2, concurrency: int = 4, transport: httpx.AsyncBaseTransport |
None = None) -> None:
        self._client = httpx.AsyncClient(base_url=base_url, timeout=timeout, transport=transport)
        self._attempts = attempts
        self._base_delay = base_delay
        self._semaphore = asyncio.Semaphore(concurrency)

    async def __aenter__(self) -> StudentApiClient:
        return self

    async def __aexit__(self, *exc_info: object) -> None:
        await self._client.aclose()

    async def request(self, method: str, path: str, **kwargs: Any) -> httpx.Response:
        """Semaphore -> retry(timeouts, connection errors, 502/503/504) -> raise for other 4xx/5xx."""
        async def once() -> httpx.Response:
            async with self._semaphore:
                response = await self._client.request(method, path, **kwargs)
                if response.status_code in RETRYABLE_STATUSES:
                    raise RetryableStatusError(response)
            return response

        response = await retry(once, attempts=self._attempts, base_delay=self._base_delay,

```

```

        retry_on=(httpx.TransportError, RetryableStatusError), label=f"{method}
{path}")
        response.raise_for_status()
        return response

    async def statistics_many(self, codes: list[str]) -> list[dict[str, Any]]:
        """Parallel HTTP requests: one Task per group, gathered; the Semaphore in request() limits
        concurrency."""
        tasks = [asyncio.create_task(self.statistics(code)) for code in codes]
        return list(await asyncio.gather(*tasks))

```

Один AsyncClient = один пул з'єднань; async with гарантує його закриття. Кожен запит проходить через семафор, потім через retry, який повторює лише транспортні помилки (timeout, розрив з'єднання) і статуси 502/503/504; решта 4xx/5xx одразу піднімаються як HTTPStatusError. Демонстрація нижче йде до справжнього Uvicorn на порту 8765: timeout 0.3 с проти endpoint-a, що спить 1 с, і /debug/flaky, який двічі відповідає 503.

```

@router.get("/flaky", summary="Перші `failures` викликів відповідають 503, далі 200")
async def flaky(request: Request, response: Response, failures: Annotated[int, Query(ge=0)] = 2) -> dict[str, int]:
    state = request.app.state
    state.flaky_calls = getattr(state, "flaky_calls", 0) + 1
    if state.flaky_calls <= failures:
        response.status_code = status.HTTP_503_SERVICE_UNAVAILABLE
    return {"call": state.flaky_calls}

```

HTTPX-клієнт проти справжнього Uvicorn (127.0.0.1:8765): паралельні запити, timeout, retry з backoff

```

-----
statistics_many -> [('ФЕП-31с', 2, 87.45), ('ФЕП-32', 0, None), ('ФЕП-41', 1, 64.0)]
[WARNING] GET /debug/slow: спроба 1/3 невдала (ReadTimeout), повтор через 0.05 с
[WARNING] GET /debug/slow: спроба 2/3 невдала (ReadTimeout), повтор через 0.10 с
[WARNING] GET /debug/slow: спроба 3/3 невдала (ReadTimeout) – здаюся
/debug/slow?seconds=1.0 з timeout=0.3 -> ReadTimeout після 3 спроб
[WARNING] GET /debug/flaky: спроба 1/3 невдала (RetryableStatusError: HTTP 503 GET /debug/flaky),
повтор через 0.05 с
[WARNING] GET /debug/flaky: спроба 2/3 невдала (RetryableStatusError: HTTP 503 GET /debug/flaky),
повтор через 0.10 с
/debug/flaky?failures=2 -> {'call': 3}

```

11. Swagger UI та виконання endpoint-ів

Student Manager API 0.8.0 OAS 3.1

/openapi.json

Асинхронний REST API системи обліку студентів (лабораторна робота №8).

Ресурси: **students** і **groups**. Дані зберігаються в SQLite через persistence layer ЛР7.

Contact Артур Лис

students CRUD, пошук за прізвищем, фільтр за групою, найкращий студент			^
GET	/students	Список студентів із пошуком, фільтром і пагінацією	▼
POST	/students	Create Student	▼
GET	/students/best	Студент із найбільшим середнім балом	▼
GET	/students/{student_id}	Get Student	▼
PATCH	/students/{student_id}	Змінити бал, перевести до іншої групи або оновити email	▼
DELETE	/students/{student_id}	Delete Student	▼
groups Статистика груп; кілька груп одночасно через asyncio.gather			^
GET	/groups	Усі групи зі статистикою	▼
GET	/groups/statistics	Статистика кількох груп одночасно (Task + gather + Semaphore)	▼
GET	/groups/{code}/students	Group Students	▼

Рис. 1. Swagger UI (/docs): OpenAPI-метадані, межу students / groups / debug

GET

/students

Список студентів із пошуком, фільтром і пагінацією

⌵

Parameters

Cancel

Name	Description
last_name string (string null) (query)	Префікс прізвища, без урахування регістру last_name minLength: 1
group string (string null) (query)	Лише студенти цієї групи ФЕП-31с pattern: ^[A-ZİİİİİA-Z]{2,5}-\d{2}[a-zİİİİİA-Z]?\$
page integer (query)	1 minimum: 1
size integer (query)	20 maximum: 100 minimum: 1
sort string (query)	id ▼

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/students?group=%D0%A4%D0%95%D0%9F-31%D1%81&page=1&size=20&sort=id' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/students?group=%D0%A4%D0%95%D0%9F-31%D1%81&page=1&size=20&sort=id
```

Server response

Code	Details
200	Response body <pre>{ "items": [{ "id": 1, "first_name": "Марта", "last_name": "Гнатишин", "full_name": "Гнатишин Марта", "group": "ФЕП-31с", "average_grade": 93.4, "email": "marta@lnu.edu.ua" }, { "id": 2, "first_name": "Оскан", "last_name": "Дзюба", "full_name": "Дзюба Оскан", "group": "ФЕП-31с", "average_grade": 81.5, "email": null }], "total": 2, "page": 1, "size": 20 }</pre> Download

Response headers

Рис. 2. GET /students?group=ФЕП-31с виконано з Swagger UI: параметри, curl, відповідь 200

Pretty-print ☐

```
[{"code": "ФЕП-31с", "students": 2, "average": 87.45, "best": {"id": 1, "first_name": "Марта", "last_name": "Г  
Марта", "group": "ФЕП-31с", "average_grade": 93.4, "email": "marta@lnu.edu.ua"}}, {"code": "ФЕП-41", "stude  
{"id": 4, "first_name": "Тарас", "last_name": "Гаврилюк", "full_name": "Гаврилюк Тарас", "group": "ФЕП-41",
```

Рис. 3. GET /groups/statistics?code=ФЕП-31с&code=ФЕП-41 у браузері

12. API-тести

Тести піднімають застосунок у тому самому процесі через `httpx.ASGITransport` (сервер не потрібен) з окремою SQLite-базою в `tmp_path`; `pytest-asyncio` в режимі `auto` виконує `async def`-тести. Зовнішній HTTP-сервіс для перевірки `retry` і `timeout` замінено `httpx.MockTransport`.

```
@pytest_asyncio.fixture
async def api(tmp_path: Path) -> AsyncIterator[httpx.AsyncClient]:
    app = create_app(database_url=f"sqlite:///{{tmp_path / 'api.db'}}.as_posix()", include_debug=True)
    transport = httpx.ASGITransport(app=app)
    async with app.router.lifespan_context(app), httpx.AsyncClient(transport=transport,
base_url="http://test") as client:
        yield client

async def test_post_returns_201_and_the_created_resource(api: httpx.AsyncClient) -> None:
    response = await api.post("/students", json=MARTA)
    assert response.status_code == 201
    assert response.json() == {"id": 1, "full_name": "Гнатишин Марта", **MARTA}

@pytest.mark.parametrize(("field", "value"), [
    ("average_grade", 150), ("average_grade", -1), ("group", "феп31"), ("first_name", " "), ("email",
"not-an-email"),
])
async def test_validation_errors_are_422(api: httpx.AsyncClient, field: str, value: object) -> None:
    response = await api.post("/students", json={**MARTA, field: value})
    assert response.status_code == 422
    assert response.json()["detail"][0]["loc"] == ["body", field]

async def test_semaphore_caps_concurrency() -> None:
    codes = [f"Г-{i}" for i in range(6)]
    limited = SlowSource(0.03)
    result = await gather_statistics(limited, codes, limit=2)
    assert [g.code for g in result] == codes and limited.peak <= 2
    unlimited = SlowSource(0.03)
```

```

    await gather_statistics(unlimited, codes, limit=6)
    assert unlimited.peak >= 3

async def test_client_retries_503_then_succeeds() -> None:
    calls: list[str] = []

    def handler(request: httpx.Request) -> httpx.Response:
        calls.append(request.url.path)
        return httpx.Response(503) if len(calls) < 3 else httpx.Response(200, json={"ok": True})

    async with StudentApiClient("http://mock", transport=httpx.MockTransport(handler), attempts=3,
base_delay=0) as client:
        assert await client.get_json("/ping") == {"ok": True}
        assert calls == ["/ping"] * 3

```

12.1. pytest -v

```

$ pytest tests/integration/test_api.py -v
tests/integration/test_api.py::test_list_is_empty_initially PASSED
tests/integration/test_api.py::test_post_returns_201_and_the_created_resource PASSED
tests/integration/test_api.py::test_validation_errors_are_422[average_grade-150] PASSED
tests/integration/test_api.py::test_validation_errors_are_422[average_grade--1] PASSED
tests/integration/test_api.py::test_validation_errors_are_422[group-\u0444\u0435\u0439\u0431\u0430] PASSED
tests/integration/test_api.py::test_validation_errors_are_422[first_name- ] PASSED
tests/integration/test_api.py::test_validation_errors_are_422[email-not-an-email] PASSED
tests/integration/test_api.py::test_unknown_field_is_rejected PASSED
tests/integration/test_api.py::test_get_missing_student_is_404_with_structured_error PASSED
tests/integration/test_api.py::test_patch_grade_then_transfer PASSED
tests/integration/test_api.py::test_delete_is_204_then_404 PASSED
tests/integration/test_api.py::test_duplicate_email_is_409 PASSED
tests/integration/test_api.py::test_search_filter_sort_and_pagination PASSED
tests/integration/test_api.py::test_best_student_overall_and_in_group PASSED
tests/integration/test_api.py::test_group_endpoints PASSED
tests/integration/test_api.py::test_semaphore_caps_concurrency PASSED
tests/integration/test_api.py::test_timeout_inside_gather PASSED
tests/integration/test_api.py::test_client_retries_503_then_succeeds PASSED
tests/integration/test_api.py::test_client_gives_up_after_attempts_on_timeout PASSED
tests/integration/test_api.py::test_client_parallel_statistics_against_the_app PASSED
===== 20 passed in 6.93s =====

$ pytest -q
213 passed in 10.41s

```

13. Перевірка типів і Git

```

$ mypy src
Success: no issues found in 76 source files

```

```

$ git log --oneline -4
c75419d lab8: API tests through ASGITransport and MockTransport, pytest-asyncio, README section, version 0.8.0
567bfd3 lab8: asyncio concurrency helpers (Task, gather, Semaphore, timeout, retry with backoff) and HTTPX client; demo and serve commands
eb4bd9c lab8: FastAPI application – Pydantic schemas, students/groups routers, store over lab-7 repositories, exception handlers
520df89 lab7: integration tests on a per-test SQLite file, README section, version 0.7.0

```

```
$ git status --short  
(чисто)
```

14. Відповіді на контрольні запитання

1. Що містить HTTP request і HTTP response?

Запит: метод, шлях із query, заголовки, тіло; відповідь: код стану, заголовки, тіло (тут JSON).

2. Для чого GET, POST, PUT, PATCH, DELETE?

GET читає, POST створює, PUT замінює ресурс цілком, PATCH змінює частину полів, DELETE видаляє.

3. Що означають 200, 201, 204, 404, 422?

Успіх; створено; успіх без тіла; ресурсу немає; дані синтаксично правильні, але не пройшли валідацію.

4. Що таке REST, resource, endpoint?

Стиль API поверх HTTP: ресурси адресуються URL, дії — методами; endpoint — конкретна пара метод + шлях.

5. Чим path parameter відрізняється від query parameter?

Path ідентифікує ресурс (/students/7), query уточнює вибірку (?group=...&page=2) і є необов'язковим.

6. Що таке FastAPI, Uvicorn і Swagger UI?

Фреймворк для async REST API з валідацією через Pydantic; ASGI-сервер, що його запускає; інтерактивна документація з OpenAPI на /docs.

7. Що таке response model і навіщо він?

Схема відповіді: FastAPI фільтрує та серіалізує лише її поля і документує їх в OpenAPI.

8. Що таке coroutine, event loop і Task?

Корутина — функція async def, що може призупинятись на await; event loop чергує їх; Task — корутина, запланована в loop і виконувана «у фоні».

9. Для чого asyncio.create_task() і gather()?

create_task запускає корутину, не чекаючи на неї; gather чекає на кілька одразу і повертає результати в порядку аргументів.

10. Що таке timeout, retry і backoff?

Максимальний час очікування; повторна спроба після збою; зростання паузи між спробами (0.2, 0.4, 0.8 с), щоб не добивати перевантажений сервіс.

11. Для чого Semaphore і навіщо обмежувати паралелізм?

Пускає не більше N задач одночасно; тисячі одночасних запитів вичерпують з'єднання, пам'ять і ліміти сервера.

12. Чим `httpx.AsyncClient` відрізняється від синхронного і чому його повторно використовують?

Методи `awaitable`, не блокують `loop`; один клієнт тримає пул `keep-alive`-з'єднань, тож створювати його на кожен запит дорого.

13. Як REST API взаємодіє з Repository Layer?

Endpoint → StudentStore → репозиторії LP7 у транзакції; API не знає SQL, репозиторії не знають HTTP.

14. Як FastAPI працює з Pydantic?

Тип параметра або тіла — Pydantic-модель; FastAPI парсить JSON, валідує, віддає 422 з деталями та генерує OpenAPI-схему.

15. Чим API integration test відрізняється від unit test?

Проходить через увесь стек (маршрутизація, валідація, store, база) реальним HTTP-запитом, а не викликає одну функцію ізольовано.

15. Висновок

Проект отримав асинхронний REST API з десятима endpoint-ами над persistence layer LP7: Pydantic-схеми з валідацією, коректні коди стану, структуровані помилки, пагінація й сортування. Асинхронна частина — статистика кількох груп через Task + gather під Semaphore і timeout, а HTTPX-клієнт додає retry з exponential backoff і керований життєвий цикл з'єднань; демонстрація проти справжнього Uvicorn показує ReadTimeout після трьох спроб і успіх після двох 503. 20 API-тестів через ASGITransport і MockTransport проходять разом з усіма 213 тестами проекту, туру у strict-режимі чистий.